

Experiment 6

AIM: Create Product Catalog Application using ASP.NET MVC Architecture. (Controllers, Models, Views, Routing)

Source: Index.cshtml

Code:

```
@model IEnumerable<Practical_6.Models.Product>
@{
    ViewBag.Title = "Product Catalog";
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-4">
    <h2 class="fw-bold"><i class="fa-solid fa-store me-2"></i>Product Catalog</h2>
    <a href="@Url.Action("Create")" class="btn btn-primary">
        <i class="fa-solid fa-plus me-1"></i>Add New Product
    </a>
</div>

<form method="get" action="@Url.Action("Index")" class="row g-2 mb-4">
    <div class="col-md-4">
        <input type="text" name="search" value="@ViewBag.Search" class="form-control"
placeholder="Search products..." />
    </div>
    <div class="col-md-4">
        <select name="category" class="form-select">
            <option value="">All Categories</option>
            @foreach (var cat in (List<string>)ViewBag.Categories)
            {
                if (cat == ViewBag.SelectedCategory)
                {
                    <option value="@cat" selected="selected">@cat</option>
                }
                else
                {
                    <option value="@cat">@cat</option>
                }
            }
        </select>
    </div>
    <div class="col-md-2">
        <button type="submit" class="btn btn-outline-secondary w-100">
            <i class="fa-solid fa-filter me-1"></i>Filter
        </button>
    </div>
    <div class="col-md-2">
        <a href="@Url.Action("Index")" class="btn btn-outline-danger w-100">Clear</a>
    </div>
</form>

@if (!Model.Any())
{
    <div class="alert alert-warning text-center">No products found.</div>
}
else
{
```

```
<div class="row g-4">
    @foreach (var product in Model)
    {
        <div class="col-sm-6 col-md-4 col-lg-3">
            <div class="card product-card h-100">
                
                <div class="card-body d-flex flex-column">
                    <span class="badge badge-category text-white mb-2 align-self-
start">@product.Category</span>
                    <h5 class="card-title">@product.Name</h5>
                    <p class="card-text text-muted small flex-grow-1">
                        @if (product.Description.Length > 70 ? product.Description.Substring(0,
70) + "..." : product.Description)
                    </p>
                    <p class="price-tag mb-1">@product.Price.ToString("C")</p>
                    <p class="small text-secondary mb-3">
                        @if (product.StockQuantity > 0)
                        {
                            <span><i class="fa-solid fa-circle-check text-success"></i> In
Stock (@product.StockQuantity)</span>
                        }
                        else
                        {
                            <span><i class="fa-solid fa-circle-xmark text-danger"></i> Out of
Stock</span>
                        }
                    </p>
                    <div class="d-flex gap-2 mt-auto">
                        <a href="@Url.Action("Details", new { id = product.Id })" class="btn
btn-sm btn-outline-primary flex-fill">View</a>
                        <a href="@Url.Action("Edit", new { id = product.Id })" class="btn btn-
sm btn-outline-secondary flex-fill">Edit</a>
                        <a href="@Url.Action("Delete", new { id = product.Id })" class="btn
btn-sm btn-outline-danger flex-fill">Delete</a>
                    </div>
                </div>
            </div>
        </div>
    }
</div>
}
```

Product.cs

```
using System.ComponentModel.DataAnnotations;

namespace Practical_6.Models
{
    public class Product
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Product name is required")]
        [StringLength(100)]
        [Display(Name = "Product Name")]
        public string Name { get; set; }

        [Required(ErrorMessage = "Description is required")]
    }
```

```
[DataType(DataType.MultilineText)]
public string Description { get; set; }

[Required]
[Range(0.01, 100000, ErrorMessage = "Price must be greater than 0")]
[DataType(DataType.Currency)]
public decimal Price { get; set; }

[Required(ErrorMessage = "Category is required")]
public string Category { get; set; }

[Display(Name = "Stock Quantity")]
[Range(0, int.MaxValue, ErrorMessage = "Stock cannot be negative")]
public int StockQuantity { get; set; }

[Display(Name = "Image URL")]
[DataType(DataType.ImageUrl)]
public string ImageUrl { get; set; }
}
}
```

ProductController.cs

```
using System.Linq;
using System.Web.Mvc;
using Practical_6.Models;

namespace Practical_6.Controllers
{
    public class ProductController : Controller
    {
        // GET: Product (also handles /Catalog/{category} route)
        public ActionResult Index(string category, string search)
        {
            var products = ProductRepository.GetAll().AsEnumerable();

            if (!string.IsNullOrEmpty(category))
            {
                products = products.Where(p => p.Category == category);
            }

            if (!string.IsNullOrEmpty(search))
            {
                products = products.Where(p => p.Name.ToLower().Contains(search.ToLower()));
            }

            ViewBag.Categories = ProductRepository.GetCategories();
            ViewBag.SelectedCategory = category;
            ViewBag.Search = search;

            return View(products.ToList());
        }

        // GET: Product/Details/5
        public ActionResult Details(int id)
        {
            var product = ProductRepository.GetById(id);
            if (product == null) return HttpNotFound();
        }
    }
}
```

```
        return View(product);
    }

    // GET: Product/Create
    public ActionResult Create()
    {
        return View();
    }

    // POST: Product/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Create(Product product)
    {
        if (ModelState.IsValid)
        {
            ProductRepository.Add(product);
            TempData["Success"] = "Product created successfully!";
            return RedirectToAction("Index");
        }
        return View(product);
    }

    // GET: Product/Edit/5
    public ActionResult Edit(int id)
    {
        var product = ProductRepository.GetById(id);
        if (product == null) return HttpNotFound();
        return View(product);
    }

    // POST: Product/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Edit(Product product)
    {
        if (ModelState.IsValid)
        {
            ProductRepository.Update(product);
            TempData["Success"] = "Product updated successfully!";
            return RedirectToAction("Index");
        }
        return View(product);
    }

    // GET: Product/Delete/5
    public ActionResult Delete(int id)
    {
        var product = ProductRepository.GetById(id);
        if (product == null) return HttpNotFound();
        return View(product);
    }

    // POST: Product/Delete/5
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public ActionResult DeleteConfirmed(int id)
    {
        ProductRepository.Delete(id);
        TempData["Success"] = "Product deleted successfully!";
        return RedirectToAction("Index");
    }
```

```
}  
}  
}
```

ProductRepository.cs

```
using System.Collections.Generic;  
using System.Linq;  
  
namespace Practical_6.Models  
{  
    // Simple in-memory "database". Swap this out for Entity Framework  
    // later (DbContext + SQL Server) without touching the Controller much.  
    public static class ProductRepository  
    {  
        private static readonly List<Product> _products;  
        private static int _nextId = 1;  
  
        static ProductRepository()  
        {  
            _products = new List<Product>();  
            Seed();  
        }  
  
        private static void Seed()  
        {  
            Add(new Product { Name = "Wireless Mouse", Description = "Ergonomic wireless mouse  
with USB receiver.", Price = 599.00m, Category = "Electronics", StockQuantity = 120, ImageUrl =  
"https://placeholder.co/400x300?text=Wireless+Mouse" });  
            Add(new Product { Name = "Mechanical Keyboard", Description = "RGB backlit mechanical  
keyboard, blue switches.", Price = 2499.00m, Category = "Electronics", StockQuantity = 45,  
ImageUrl = "https://placeholder.co/400x300?text=Keyboard" });  
            Add(new Product { Name = "Running Shoes", Description = "Lightweight breathable  
running shoes for daily training.", Price = 1899.00m, Category = "Footwear", StockQuantity = 60,  
ImageUrl = "https://placeholder.co/400x300?text=Running+Shoes" });  
            Add(new Product { Name = "Ceramic Coffee Mug", Description = "350ml ceramic mug,  
microwave and dishwasher safe.", Price = 249.00m, Category = "Home & Kitchen", StockQuantity =  
200, ImageUrl = "https://placeholder.co/400x300?text=Coffee+Mug" });  
            Add(new Product { Name = "Yoga Mat", Description = "Non-slip yoga mat, 6mm thickness,  
includes carry strap.", Price = 799.00m, Category = "Fitness", StockQuantity = 0, ImageUrl =  
"https://placeholder.co/400x300?text=Yoga+Mat" });  
        }  
  
        public static List<Product> GetAll()  
        {  
            return _products;  
        }  
  
        public static Product GetById(int id)  
        {  
            return _products.FirstOrDefault(p => p.Id == id);  
        }  
  
        public static void Add(Product product)  
        {  
            product.Id = _nextId++;  
            _products.Add(product);  
        }  
    }  
}
```

```

public static void Update(Product product)
{
    var existing = GetById(product.Id);
    if (existing == null) return;

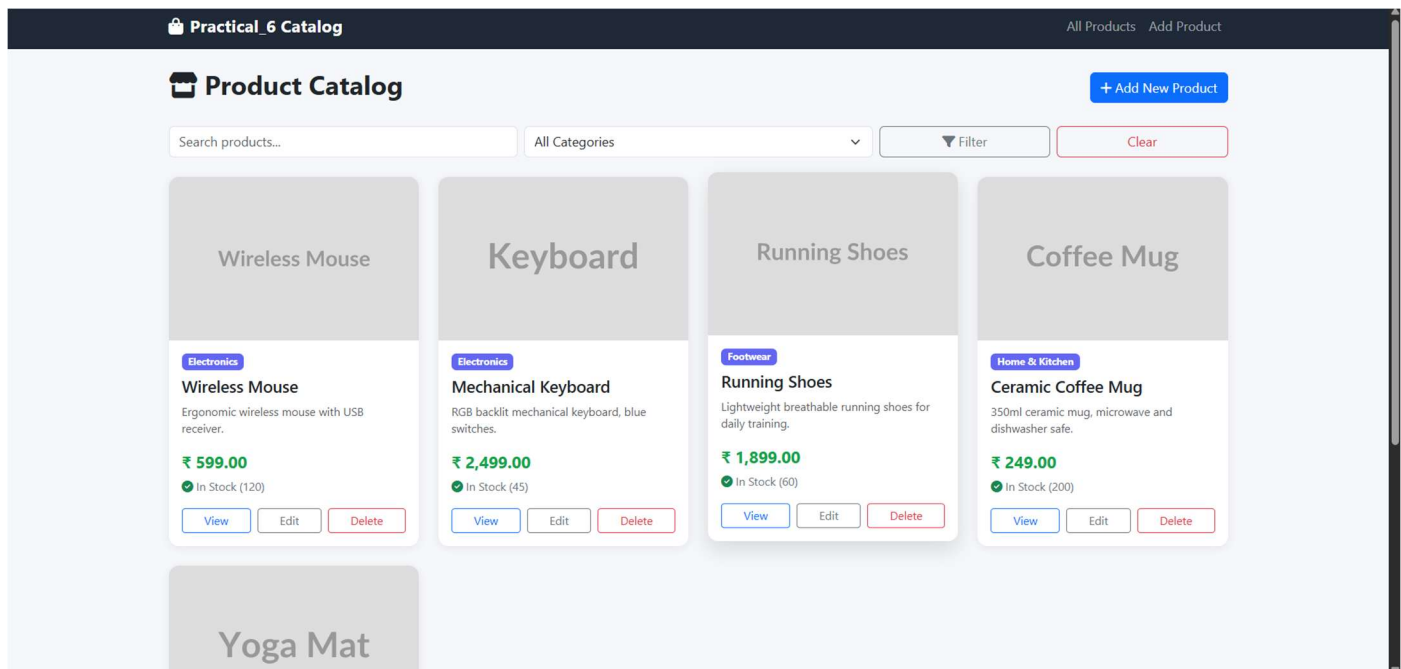
    existing.Name = product.Name;
    existing.Description = product.Description;
    existing.Price = product.Price;
    existing.Category = product.Category;
    existing.StockQuantity = product.StockQuantity;
    existing.ImageUrl = product.ImageUrl;
}

public static void Delete(int id)
{
    var existing = GetById(id);
    if (existing != null)
    {
        _products.Remove(existing);
    }
}

public static List<string> GetCategories()
{
    return _products.Select(p => p.Category).Distinct().OrderBy(c => c).ToList();
}
}

```

Output:



 **Practical_6 Catalog**

All ProductsAdd Product

Keyboard

Electronics

Mechanical Keyboard

₹ 2,499.00

RGB backlit mechanical keyboard, blue switches.

Stock: 45 units

Edit

Delete

← Back

© 2026 Practical_6 — Product Catalog Application (ASP.NET MVC)

 **Practical_6 Catalog**

All ProductsAdd Product

 **Edit Product**

Product Name

Mechanical Keyboard

Description

RGB backlit mechanical keyboard, blue switches.

Price

2699.00

Stock Quantity

45

Category

Electronics

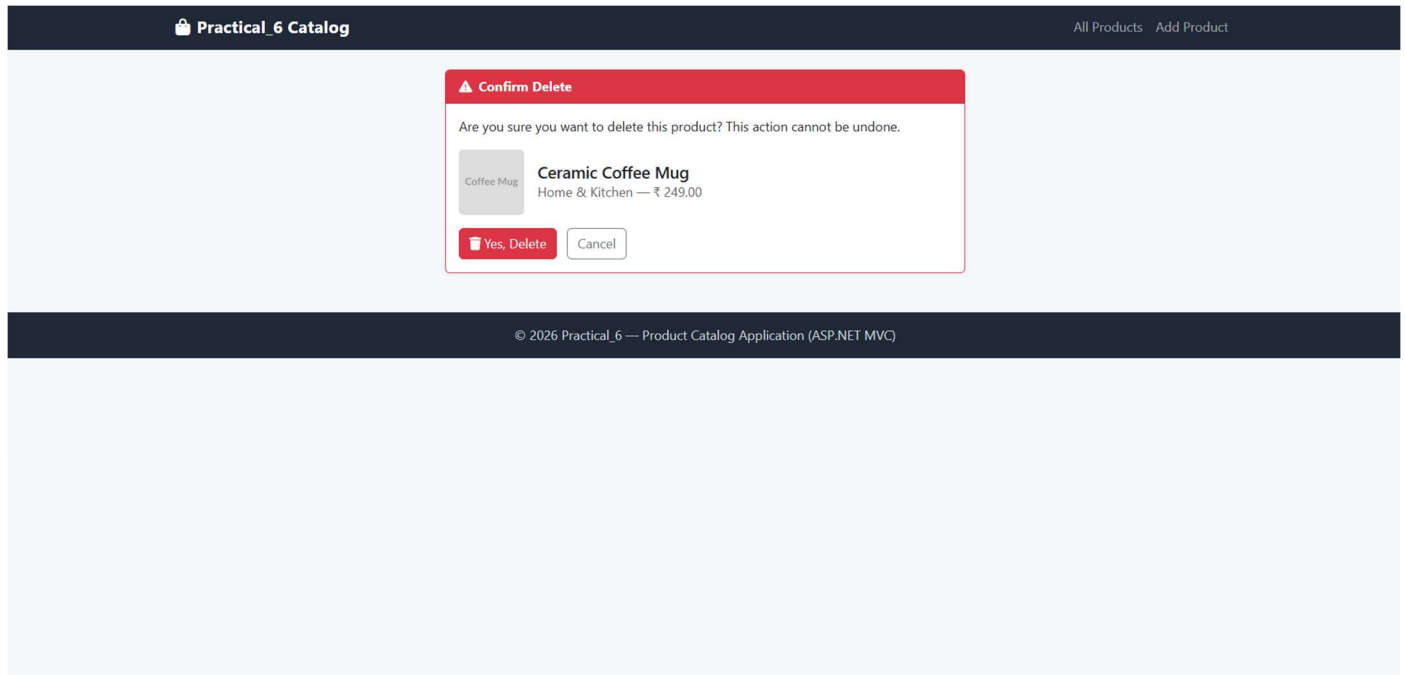
Image URL

https://placeholder.co/400x300?text=Keyboard

Update Product

Cancel

© 2026 Practical_6 — Product Catalog Application (ASP.NET MVC)



GitHub URL: <https://github.com/jainishkakkad133582/.Net-Practicals/tree/main/Practical%206>

GitHub URL: <https://github.com/jainishkakkad133582/.Net-Practicals/tree/main>